LESSON 2 · COMPUTER FUNDAMENTALS TRACK

How the Internet Actually Works

What really happens between typing a URL and a page loading on your screen

Last lesson, we went inside a single machine — CPU, RAM, binary, bytecode. Today we zoom out to what happens when that machine talks to another one, anywhere in the world, in under a second. Every website you've ever visited, every API call your own projects have made, every M-Pesa or card payment confirmation that pings back to an app — it all runs on the same handful of ideas we're covering today.

> **Why this lesson exists**
>
> You can build with APIs and call requests.get(url) for years without knowing what actually happens after you press enter. That's fine, until a request times out, a payment webhook doesn't fire, or someone asks why a site shows “connection not secure.” This lesson is what's actually happening in that gap between request and response.

What you'll walk away with

- What actually happens, step by step, between typing a URL and seeing a page load.
- How a name like google.com becomes an address a computer can actually connect to.
- What an HTTP request and response really contain, and what status codes like 404 and 500 actually mean.
- Why your data doesn't travel as one big chunk, and what a packet actually is.
- What a server, a port, and an API really are underneath the terms.
- What HTTPS actually protects, and why it matters most on payment and login forms.

Session Plan — 2 Hour Class

Same format as Lesson 1: suggested pacing below, detailed talking points and demos in each section that follows.

Time	Segment	Focus
5 min	Welcome & framing	Why this lesson matters, what you'll walk away with
20 min	§1 Names & Addresses	IP addresses, domain names, how DNS resolves them

20 min	§2 Request & Response	Client-server model, HTTP methods, status codes, live demo
10 min	Break	—
20 min	§3 Packets & Routing	TCP/IP, how data is split and routed, traceroute demo
15 min	§4 Servers, Ports & APIs	What a server actually is, ports, real-world API example
15 min	§5 HTTPS & Encryption	What the padlock icon actually protects, and why it matters
15 min	Wrap-up & Q&A;	Key terms recap, homework, next lesson preview

TIME: 20 MIN

1. Names vs. Addresses: IP Addresses & DNS

Every device connected to the internet — your laptop, your phone, the server hosting a website — has a unique numerical address called an **IP address**, something like `142.250.183.14`. Computers are perfectly happy using these numbers to find each other. Humans are not — nobody wants to memorize a string of numbers for every website they visit. That gap is exactly what domain names and DNS exist to solve.

The phone book analogy

Think of an IP address like a phone number, and a domain name like a contact's name saved in your phone. You tap “Mum”, not her actual number — your phone looks the number up for you, invisibly. **DNS (Domain Name System)** does exactly this for the internet: you type `google.com`, and somewhere behind the scenes, DNS looks up the actual IP address behind that name.

How a DNS lookup actually happens

Step 1 — Your device asks

Your browser asks a DNS resolver (usually run by your internet provider) “what's the IP address for google.com?”

Step 2 — Cache check

The resolver first checks whether it already knows the answer from a recent lookup. If so, it replies immediately — this is why the second visit to a site often feels faster.

Step 3 — Root & TLD servers

If not cached, the resolver asks a chain of servers — starting with root servers, then servers responsible for “.com” — each pointing it closer to the answer.

Step 4 — Authoritative answer

Eventually the resolver reaches the authoritative server for google.com, which gives back the actual IP address.

Step 5 — Connection

Your browser now has an IP address and can actually open a connection to that server.

See it for yourself

You can look up the IP address behind any domain yourself, right now, from a terminal:

```
$ nslookup google.com
Server:   8.8.8.8
Address:  8.8.8.8#53

Non-authoritative answer:
Name:    google.com
Address: 142.250.183.14
```

Or from inside Python, using the built-in socket module:

```
>>> import socket
>>> socket.gethostbyname('google.com')
'142.250.183.14'
```

> **Quick check**
>
> If DNS servers went down everywhere for an hour, would the internet actually stop working? *(Answer: mostly yes, for humans — but if you already knew a site's IP address, you could still connect directly. This is exactly why DNS is sometimes called a convenience layer, not the network itself.)*

TIME: 20 MIN

2. The Request-Response Cycle

Once your browser has an IP address, it needs a shared language to actually ask that server for something. That language is **HTTP** — HyperText Transfer Protocol. The whole model rests on one relationship: the **client** (your browser, or your own Python script) asks for something, and the **server** answers.

Anatomy of an HTTP request

- **Method** — what kind of action is being requested. GET asks for data. POST sends new data to be created or processed.
- **Path** — which specific resource on the server, e.g. `/products/42`.
- **Headers** — metadata about the request: what type of content the client accepts, authentication tokens, and so on.
- **Body** — the actual data being sent, present on requests like POST (e.g. the form data when you submit a login).

Anatomy of an HTTP response

- **Status code** — a 3-digit number telling you what happened. We'll look at these next.
- **Headers** — metadata about the response: content type, caching rules, and more.
- **Body** — the actual content: HTML for a webpage, JSON for an API, and so on.

Code	Meaning	What it actually tells you
200	OK	The request succeeded — here's your data
301 / 302	Redirect	This moved — go look over there instead
404	Not Found	The server exists, but nothing lives at that path
401 / 403	Unauthorized / Forbidden	You need to log in, or you're not allowed here
500	Internal Server Error	Something broke on the server's side, not yours

See it for yourself

Let's actually make a request and look at the real response, live, using Python's popular `requests` library.

```
>>> import requests
>>> response = requests.get('https://api.github.com')
>>> response.status_code
200
>>> response.headers['content-type']
'application/json; charset=utf-8'
>>> response.json()['current_user_url']
'https://api.github.com/user'
```

That single line, `requests.get(...)`, is doing everything we just described: a DNS lookup for api.github.com, opening a connection, sending an HTTP GET request, and receiving back a status code, headers, and a body.

> **Why this matters for you as a developer**
>
> The next time an API call in your own code returns a 404, you'll know exactly what that means: your request reached the server fine, but you asked for a path that doesn't exist. A 500 means the opposite — your request was fine, but something broke on their end. That distinction changes where you go looking to fix the bug.

TIME: 20 MIN

3. Data Travels in Packets

Here's something that surprises most people: your data doesn't travel across the internet as one continuous stream. It gets broken into small chunks called **packets**, each sent separately, sometimes over completely different physical routes, and reassembled in the correct order once they all arrive.

The envelope analogy

Imagine mailing a long letter by tearing it into a dozen pieces, putting each piece in its own numbered envelope, and mailing them separately — possibly through different sorting offices. As long as the pieces are numbered, the recipient can reassemble them in the right order even if envelope 7 arrives before envelope 3. That's essentially what **TCP** (Transmission Control Protocol) does with packets — it numbers them, confirms each one arrived, and asks for a resend if one goes missing.

TCP vs. UDP

	TCP	UDP
Delivery	Reliable — confirms every packet arrives, resends if not	Best-effort — no confirmation, no resending
Order	Guaranteed — packets reassembled in the right order	Not guaranteed
Speed	Slightly slower due to all the checking	Faster — no overhead from confirmations
Used for	Web pages, APIs, file downloads — anywhere accuracy matters	Video calls, live streaming — where speed matters more than a perfect copy

This is why a slightly glitchy video call is normal — a dropped frame here and there is an acceptable trade for speed — but you'd never accept a webpage or a bank transfer arriving with random missing pieces. Different problems call for different delivery guarantees, and that's exactly why both protocols exist side by side.

See it for yourself

You can actually watch your packets hop across the network, router by router, using the traceroute command (on Windows, it's `tracert`):

```
$ traceroute google.com
 1  192.168.1.1        2.114 ms
 2  41.203.XXX.XXX     8.902 ms   <- your ISP
 3  196.XX.XX.XX      14.221 ms
 4  ...
 9  142.250.183.14    31.556 ms  <- destination reached
```

Every one of those numbered rows is a router your data physically passed through. A request that feels instant is quietly hopping across many machines to get there.

TIME: 15 MIN

4. Servers, Ports, and APIs

We've used the word “server” a lot — let's make it concrete. A server is not necessarily special hardware. It's simply **a program that's listening for incoming requests** and knows how to respond to them. The same laptop you code on could run a server right now, on your own machine.

Ports — the apartment number, not just the building

If an IP address is like a building's street address, a **port** is the specific apartment number within that building. A single server can run many programs at once, each listening on a different port. Port 80 is the standard, well-known door for plain HTTP traffic; port 443 is the equivalent for HTTPS. When your browser connects to `https://example.com`, it's really connecting to that server's IP address, on port 443, by default — you just never see the port number because it's the expected default.

APIs — servers built for programs, not people

A regular website returns HTML meant for a human to read in a browser. An **API** (Application Programming Interface) is the same request-response idea, but built for one program to talk to another — usually returning structured data like JSON instead of a formatted page.

> **A familiar example**
>
> Think about an M-Pesa or card payment confirmation inside an app. When a payment completes, the payment provider's server sends a request to the app's own server — this is called a **webhook**, an API call triggered by an event rather than a person clicking something. The app's server receives that request, checks the payment details, and updates the order status accordingly. Same request-response model we've covered all lesson — just one server talking to another, instead of a human clicking a browser.

See it for yourself

You can see which ports are open and listening on your own machine:

```
$ lsof -i -P | grep LISTEN
python3   1421  essa   4u  IPv4  TCP  *:8000 (LISTEN)
node      2033  essa   6u  IPv4  TCP  *:3000 (LISTEN)
```

If you've ever run a local development server on port 3000 or 8000 while building a project, this is exactly what you were doing — running your own tiny server, listening on a port, waiting for requests.

TIME: 15 MIN

5. A Layer of Trust: HTTPS & Encryption

Remember all those routers your packets hop through on their way to a destination? Every one of those hops is a machine that technically could look at your data as it passes through — unless that data is encrypted. That's exactly what the “S” in **HTTPS** adds: **encryption**, so that even if a packet is intercepted mid-journey, it's unreadable without the right key.

What actually happens before the padlock appears

Before any HTTP request is even sent over HTTPS, your browser and the server perform what's called a **TLS handshake** — a brief exchange where they agree on an encryption method and exchange keys, without ever sending the actual key in a way that could be intercepted. Only after that handshake completes does your actual request — including anything sensitive like a password or card number — get sent, now scrambled into unreadable ciphertext to anyone except the two endpoints.

> **Why this matters for you, practically**
>
> Never enter a password or payment details on a site that shows “Not Secure” or lacks the padlock icon — without HTTPS, that data can travel as plain, readable text through every hop along the way. This is exactly why payment providers like Flutterwave and M-Pesa's API integrations require HTTPS endpoints — it's not a formality, it's the only thing standing between a card number and anyone watching the network.

Discuss with your instructor

- If HTTPS encrypts the data in transit, does that mean a website itself can't be storing your password insecurely on their end? Why or why not?
- Have you ever seen a browser warning about a site's certificate? What do you think that warning is actually protecting you from?

6. Summary: The Full Journey

Let's put the entire lesson together, start to finish. You type a URL -> DNS translates the domain name into an IP address -> your browser opens a connection to that address, on the right port -> if it's HTTPS, a TLS handshake happens first -> your browser sends an HTTP request, broken into packets, routed across possibly dozens of hops -> the server processes the request and sends back a response, as packets, the same way -> your browser reassembles it and renders the page. All of this, start to finish, usually happens in well under a second.

> **Key terms from this lesson**

> IP address · Domain name · DNS · Root & TLD servers · Client-server model · HTTP · Request / response · Method (GET/POST) · Status code · TCP · UDP · Packet · Router · Traceroute · Server · Port · API · Webhook · HTTPS · TLS handshake · Encryption

Before your next meetup

- Run `nslookup` (or `socket.gethostbyname()` in Python) on three of your favorite websites and note the IP addresses.
- Run `requests.get()` against a public API of your choice and print the status code and one field from the response.
- Run `traceroute` (or `tracert` on Windows) to a site and count how many hops your data takes.
- Write down, in your own words (2–3 sentences), what the padlock icon in your browser actually guarantees — and what it doesn't.

Next lesson: How Databases Actually Store Your Data — what really happens to information after your program hits 'save.'